

# PyQt5

This document contains useful information on how to create a simple GUI using  
PyQt5

Note: The terminology included in this document and the ability to trace through examples will be  
expected on exams or quizzes.

Python, like most coding languages, is constantly evolving with new modules being added and existing modules being  
modified. This handout provides a basic understanding; however, true mastery requires practice and your own exploration  
of what python has to offer.

# Table of Contents

|                                 |           |
|---------------------------------|-----------|
| <b>Introduction</b>             | <b>2</b>  |
| <b>Terminology</b>              | <b>2</b>  |
| <b>Documentation</b>            | <b>2</b>  |
| <b>Hello PyQt!</b>              | <b>3</b>  |
| <b>Basic PyQt App Outline</b>   | <b>3</b>  |
| <b>Window Creation</b>          | <b>3</b>  |
| <b>Inheritance</b>              | <b>4</b>  |
| <b>Window Customization</b>     | <b>4</b>  |
| <b>Layouts</b>                  | <b>5</b>  |
| QHBoxLayout                     | 5         |
| QVBoxLayout                     | 5         |
| Nested Layouts                  | 6         |
| Additional Layouts              | 8         |
| <b>Labels</b>                   | <b>8</b>  |
| <b>Textboxes</b>                | <b>9</b>  |
| <b>Buttons</b>                  | <b>9</b>  |
| <b>Handling Events</b>          | <b>10</b> |
| <b>Sender Method</b>            | <b>12</b> |
| <b>Displaying Tabular Data</b>  | <b>13</b> |
| QDialog                         | 13        |
| SimpleTableModel                | 13        |
| QTableView                      | 13        |
| General Outline                 | 14        |
| <b>Documentation References</b> | <b>15</b> |

Python, like most coding languages, is constantly evolving with new modules being added and existing modules being modified. This handout provides a basic understanding; however, true mastery requires practice and your own exploration of what python has to offer.

# Introduction

A Graphical User Interface (**GUI**) is simply a stand-alone application that a user can interact with. GUI is an umbrella term, with examples that range from Google Chrome to Sublime and even the simple calculator installed on your computer. In this course, we will expose you to the PyQt5 Library which enables you to create GUIs using Python. In order to start using the PyQt5 library, install PyQt5 by executing `conda install pyqt` from the command line.

## Terminology

**Application** - An object that handles GUI initialization and finalization

**Widget** - An object used within an application that provides functionality to the user. Some common examples are text boxes, check boxes, labels, buttons, and layouts.

**Parent Widget** - A widget used to organize sub widgets within a given window. For example, a layout can organize labels and text boxes, therefore the layout widget is the parent widget.

**Child Widget** - A widget embedded in a parent widget. Using the previous example, the labels and text boxes are the child widget because they are embedded within the layout

**Event** - An action or occurrence recognized by software, often originating asynchronously from the external environment, that may be handled by the software. For example, a user presses the next button which triggers an event and the application changes pages.

**Layout** - Special widgets that organize child widgets to display information in a structured manner

**Positioning** - The orientation of an object within a GUI. For example, the main window size, the location of a button, etc.

**Signal** - Emitted when something of potential interest happen

**Slot** - a Python callable (usually a function that a signal is connected to)

**Absolute Positioning** - Structuring objects such that their positions are static and do not change with the environment

**Relative Positioning** - Structuring objects such that their positions are dynamic and change with the environment

## Documentation

We recommend that you use this [link](#) to the PySide6 documentation. There are subtle differences between PyQt5 and PySide6 that we will not discuss in this course, most of the documentation and classes are the same. For a comprehensive list of missing bindings visit [this website](#). If you would like to refer to the official PyQt5s documentation [click here](#).

Python, like most coding languages, is constantly evolving with new modules being added and existing modules being modified. This handout provides a basic understanding; however, true mastery requires practice and your own exploration of what python has to offer.

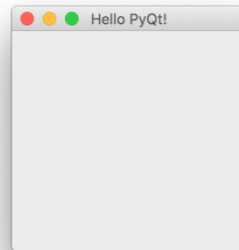
# Hello PyQt!

Below is an example of a simple GUI using PyQt5. Note that this is a **simplified** version of the GUI that we will expect you to write. GUIs typically use inheritance, which is shown farther below.

```
import sys
from PyQt5.QtWidgets import
    QApplication, QWidget

app = QApplication(sys.argv)

w = QWidget()
w.setWindowTitle("Hello PyQt!")
w.show()
return_code = app.exec_()
sys.exit(return_code)
```



To start, we must first import `sys` then import the `QApplication` and `QWidget` constructors from the `PyQt5.QtWidgets` module. The `QApplication` object serves as a foundation for our application, specifically managing the applications control flow and main settings. After initializing the `QApplication` object we then build the window of the application using the `QWidget` constructor. The `QWidget` class is the atom of the user interface; it receives mouse, keyboard, and other events from the application and paints a representation of itself on the screen. After initializing the `QWidget`, we can call a method on the `QWidget` object to set the window title to “Hello PyQt!” to decorate our interface. Once we finish customizing the window, we can show our application by calling the `show` method on the `QWidget` object. Finally, we execute the application and pass the return code to the `sys.exit` function so that our GUI closes properly.

## Basic PyQt App Outline

1. Create a `QApplication` object
2. Create a main application window (`QWidget` object)
3. Set parameters of the main window
  - a. Create and add child widgets and main window configuration
4. Show main application window
5. Start the app with `app.exec_()`

## Window Creation

A window is a widget that typically has a frame and a title bar. In general, programmers add child widgets to the window/parent widgets frame to add additional functionality and decoration to the GUI. Child widgets determine the functionality of the GUI and are usually contained in a layout. Within the PyQt5 Library, the `PyQt5.QtWidgets` module offers several classes that can be used within an

Python, like most coding languages, is constantly evolving with new modules being added and existing modules being modified. This handout provides a basic understanding; however, true mastery requires practice and your own exploration of what python has to offer.

application. For this course, we will primarily use the QWidget class. However, classes like QDialog, QMessageBox, and QAbstractTableModel may appear within GUIs we use in lecture, recitation, or homework assignments.

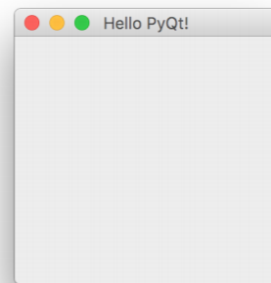
## Inheritance

In the Hello PyQt example above, we have code that generates one window with a title. In this simple case we did not need to write a class because of the task's simplicity. As GUIs scale, the previous structure will not suffice, and it will be necessary to write our own classes to create more elaborate and useful applications. When writing our own classes, it is important to use descriptive class and variable names so other developers can maintain your code if needed. Now, let's revisit the Hello PyQt example and rewrite an equivalent application using our own class.

```
import sys
from PyQt5.QtWidgets import
    QApplication, QWidget

class MainWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Hello PyQt!")

if __name__ == "__main__":
    app = QApplication(sys.argv)
    main = MainWindow()
    main.show()
    sys.exit(app.exec_())
```



When the above file is executed as a script, the `__name__` variable takes a value of `"__main__"`, therefore the if statement is entered. Within the if statement we construct our application object just like before, then initialize a `MainWindow` object. When the `MainWindow` object is initialized, the `MainWindow __init__` method is called, which then calls the `QWidget __init__` method using the `super` keyword. Now we have a `MainWindow` object that inherits all the methods and attributes of the `QWidget` class, thus we can call methods like `setWindowTitle` on our `MainWindow` object. After customizing the `MainWindow`, we finally show the window and execute the app just like before.

## Window Customization

Now that we know how to define a class that extends a widget, we can focus on creating more extensive applications by customizing windows. Typically the first step to customizing a window is to determine the layout that fits a particular window's objectives. After constructing a layout, child widgets are added to the layout to enable additional functionality. After all the widgets are added to the window, the final step of customization involves handling events by writing methods.

Python, like most coding languages, is constantly evolving with new modules being added and existing modules being modified. This handout provides a basic understanding; however, true mastery requires practice and your own exploration of what python has to offer.

## Layouts

When customizing a window, one can use either absolute positioning or relative positioning. For the GUIs we will make in this course we **require** that you use relative positioning. This means that the positions of the widgets displayed within the window are dynamic and will change with the window size instead of being fixed to a specific coordinate. The classes that implement relative positioning that we will use for this course are QHBoxLayout and QVBoxLayout.

### QHBoxLayout

The QHBoxLayout widget allows developers to add child widgets to the layout instance such that each child widget is aligned horizontally. This concept can be easily visualized by extending our previous example by adding buttons with labels, “Hello” and “Hola”.

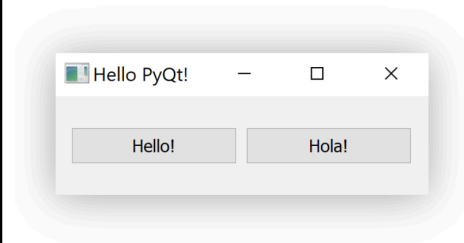
```
import sys
from PyQt5.QtWidgets import QApplication,
QWidget, QHBoxLayout, QPushButton

class MainWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Hello PyQt!")

        hbox = QHBoxLayout()
        hbox.addWidget(QPushButton("Hello!"))
        hbox.addWidget(QPushButton("Hola!"))

        self.setLayout(hbox)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    main = MainWindow()
    main.show()
    sys.exit(app.exec_())
```



Notice that the order matters. For example if we added QPushButton(“Hola”) before QPushButton(“Hello”) the buttons would be displayed in reversed order. Also notice that we call the .addWidget method on the hbox layout to add the button widgets to the hbox.

### QVBoxLayout

The QVBoxLayout widget allows developers to add child widgets to the layout instance such that each child widget is aligned vertically. This concept can be easily visualized by changing our previous example.

```

import sys
from PyQt5.QtWidgets import QApplication,
QWidget, QVBoxLayout, QPushButton

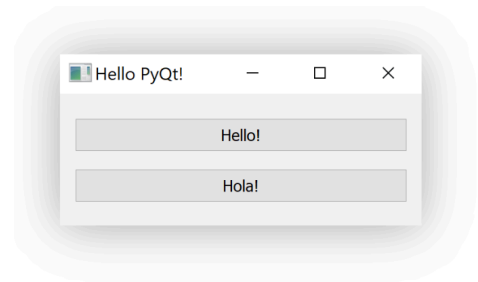
class MainWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Hello PyQt!")

        vbox = QVBoxLayout()
        vbox.addWidget(QPushButton("Hello!"))
        vbox.addWidget(QPushButton("Hola!"))

        self.setLayout(vbox)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    main = MainWindow()
    main.show()
    sys.exit(app.exec_())

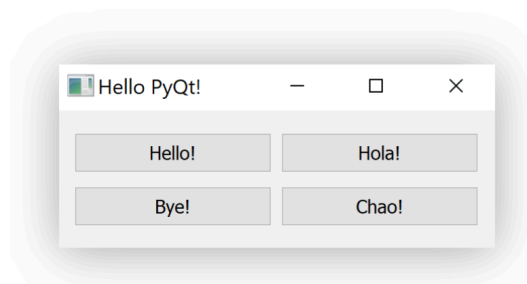
```



Notice that the order matters. For example if we added `QPushButton("Hola")` before `QPushButton("Hello")` the buttons would be displayed in reverse order. Also notice that we call the `.addWidget` method on the `vbox` layout to add the button widgets to the `vbox`.

## Nested Layouts

Consider the following window.



The window above can be constructed using both `vboxes` and `hboxes`. There are two ways to accomplish this, consider the following code for the first example.

Python, like most coding languages, is constantly evolving with new modules being added and existing modules being modified. This handout provides a basic understanding; however, true mastery requires practice and your own exploration of what python has to offer.

```

class MainWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Hello PyQt!")

        vbox = QVBoxLayout()

        hbox1 = QHBoxLayout()
        hbox1.addWidget(QPushButton("Hello!"))
        hbox1.addWidget(QPushButton("Hola!"))

        hbox2 = QHBoxLayout()
        hbox2.addWidget(QPushButton("Bye!"))
        hbox2.addWidget(QPushButton("Chao!"))

        vbox.addLayout(hbox1)
        vbox.addLayout(hbox2)

        self.setLayout(vbox)

```

In the code above, we create two hboxes acting like rows for our window then stack them vertically using a vbox. The first row / hbox contains the buttons “Hello!” and “Hola!” and the second row / hbox contains the buttons “Bye!” and “Chao!”. When adding the hboxes to the main vbox layout, we can call the .addLayout method on the vbox layout to add each row / hbox to the main vbox layout. Finally, we set the layout of MainWindow to the outer vbox using QWidget’s .setLayout method.

```

class MainWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Hello PyQt!")

        hbox = QHBoxLayout()

        vbox1 = QVBoxLayout()
        vbox1.addWidget(QPushButton("Hello!"))
        vbox1.addWidget(QPushButton("Bye!"))

        vbox2 = QVBoxLayout()
        vbox2.addWidget(QPushButton("Hola!"))
        vbox2.addWidget(QPushButton("Chao!"))

        hbox.addLayout(vbox1)
        hbox.addLayout(vbox2)

        self.setLayout(hbox)

```

In the code above, we can use two vboxes acting like columns for our window then stack them horizontally using an hbox. The first column / vbox contains the buttons “Hello!” and “Bye!” and the second column / vbox contains the buttons “Hola!” and “Chao!”. When adding the vboxes to the main hbox layout, we can call the .addLayout method on the hbox layout to add each column / vbox to the main vbox layout. Finally, we set the layout of MainWindow to the outer hbox using QWidget’s .setLayout method.

Python, like most coding languages, is constantly evolving with new modules being added and existing modules being modified. This handout provides a basic understanding; however, true mastery requires practice and your own exploration of what python has to offer.

Alternatively, if you are creating a widget instead of a layout and trying to set that as the central widget, you can use a similar function to `self.setLayout` called `self.setCentralWidget(self.myWidget)` and then apply whatever widget you would like to be the main display widget.

Although in this example we are only adding buttons to our window, the concept of nesting layouts scales to create robust and dynamic applications.

## Additional Layouts

Oftentimes, it may be helpful to use a combination of several layouts when creating an application. Here are a couple we find useful, for further details see the links to the documentation below.

- [QFormLayout](#)
- [QGridLayout](#)
- [QGroupBox](#)

You are not expected to know these additional layouts for the exams.

# Basic PyQt5 Objects

## Labels

Labels are used for displaying text or an image. No user interaction functionality is provided. The visual appearance of the label can be configured in various ways. To create a textbox, we simply create a `QLabel()` object. There are three main methods that are used with labels - `text()`, `setText()`, and `setAlignment()`.

### `text()`

This method retrieves the text written inside of the label.

### `setText()`

This method sets the text inside a label.

I am a label

A label in its most basic form just appears as written text on the GUI.

### `setFont()`

This method allows us to set a font. It takes in `QFont(font_name, size)` as a parameter, where `font_name` is the name of the font as a string and `size` is the size of the font as an integer.

## setAlignment()

This method aligns the text as per alignment constants. Examples include Qt.AlignCenter, Qt.AlignLeft, and Qt.AlignRight. These alignment constants can be accessed by importing Qt like such:

```
from PyQt5 import QtCore
```

To center align a label, we can simply do

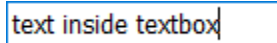
```
label.setAlignment(QtCore.Qt.AlignCenter)
```

## Textboxes

Textboxes are used to take in user input to create an output. A classic example is the search bar in a search engine. To create a textbox, we simply create a QLineEdit() object. There are two main methods that are used with textboxes - text() and setText().

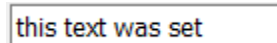
### text()

This method retrieves the text written inside of the textbox.

A screenshot of a text box with a blue border. Inside the box, the text "text inside textbox" is written in a black, monospaced font. The cursor is at the end of the text.

### setText()

This method sets the text inside a textbox.

A screenshot of a text box with a blue border. Inside the box, the text "this text was set" is written in a black, monospaced font. The cursor is at the end of the text.

There is no visual difference in the final output between the two methods.

## clear()

This method removes all text from within a textbox.

## Buttons

Buttons allow us to connect inputs with desired outputs and are commonly used in all kinds of applications. Buttons are typically connected to events, which usually occur when the button is clicked. In PyQt5, we will explore a few button methods and their functions.

### Creating a Button

To create a button, we simply create a QPushButton() object. We may pass in a string as a parameter that we want to be displayed on the button.

Python, like most coding languages, is constantly evolving with new modules being added and existing modules being modified. This handout provides a basic understanding; however, true mastery requires practice and your own exploration of what python has to offer.

## Enabling/Disabling a Button

Buttons are automatically enabled upon creation. We can enable or disable them by using the `setEnabled()` method and passing in a `True` or `False` into the parameter. `True` would enable the button while `False` will disable the button. We often use this function in accordance with the rest of the GUI to check that the user has filled in the appropriate text boxes before being able to click on the button, ensuring that we don't get a null error.



## Radio Buttons

Radio buttons are a type of button that are used to select one choice from multiple options. This button is in the selected state when the circle is filled and deselected when the circle is empty. To create a radio button, we simply create a `QRadioButton()` object. The most important method you need to know for this button is the `isChecked()` method, which will help you determine which button is checked.



Only one radio button can be selected at a time.

## Handling Events

An event is an action or occurrence recognized by software, often originating asynchronously from the external environment, that may be handled by the software. For example, a user presses the next button which triggers an event and the application changes pages. To handle our first event, let's revisit the previous example and print "Hi !" to the command line when the "Hello" button is pressed. We can do this by using either a lambda function,

```
class MainWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Hello PyQt!")

        hbox = QHBoxLayout()
        self.button = QPushButton("Hello!")
        self.button.clicked.connect(lambda x : print("Hi"))
        hbox.addWidget(self.button)

        self.setLayout(hbox)
```

Or a method,

Python, like most coding languages, is constantly evolving with new modules being added and existing modules being modified. This handout provides a basic understanding; however, true mastery requires practice and your own exploration of what python has to offer.

```

class MainWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Hello PyQt!")

        hbox = QHBoxLayout()
        self.button = QPushButton("Hello!")
        self.button.clicked.connect(self.print_hi)
        hbox.addWidget(self.button)

        self.setLayout(hbox)

    def print_hi(self):
        print("Hi!")

```

Notice the new line of code that allows the button to handle the event in this example:  
**self.button.clicked.connect(self.print\_hi)**

This code tells the GUI program that whenever the button is clicked, it needs to connect that button to the **print\_hi** instance method we defined. Therefore, each time we click on the button, the **print\_hi** method gets called and prints out "Hi!" on the command line.

The general syntax for event handling is the following:

**self.<widget\_name>.<user\_action>.connect(<slot or event\_handling\_function>)**

User actions that may be handled are dependent upon the specific widget. For example, the **QPushButton** recognizes **clicked** as a handled user action, while the **QLineEdit** widget includes **returnPressed** and **textEdited**, which are used when enter is pressed and the text is edited, respectively.

Most of the time, when handling events you want to do something more advanced than printing to the command line. If an event should perform a simple task then using a lambda function is appropriate, otherwise it's best to write a method. Within the method definition it is common to use conditionals and call widget-specific methods to add, modify, or retrieve information from the widget. For example, if a user enters their name into a **QLineEdit** object and you want to capture this information after an event is handled then you should call the appropriate method (**text**) on the **QLineEdit** instance to get the current text. It is important to note that all of the objects we will use are child classes to other classes therefore when reading documentation it may be useful to reference the parent class's documentation to understand all the inherited attributes and methods.

Python, like most coding languages, is constantly evolving with new modules being added and existing modules being modified. This handout provides a basic understanding; however, true mastery requires practice and your own exploration of what python has to offer.

## Sender Method

If multiple widgets are going to be connected to the same event, the `sender()` method can be used to distinguish which widget was clicked, toggled, etc.. Consider the following event handler where `btn1`, `btn2`, and `btn3` are all `QPushButton`s which are connected to `on_button_clicked()`:

```
class MainWindow (QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Test GUI")

        vbox = QVBoxLayout()
        hbox = QHBoxLayout()

        self.b1 = QPushButton("Button 1")
        self.b2 = QPushButton("Button 2")
        self.b3 = QPushButton("Button 3")

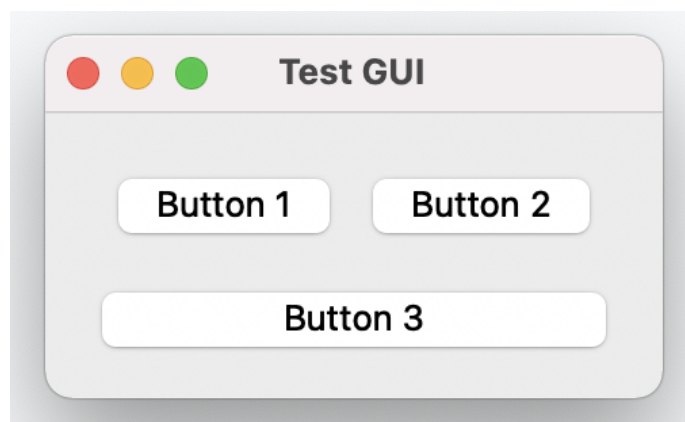
        hbox.addWidget(self.b1)
        hbox.addWidget(self.b2)
        vbox.addLayout(hbox)
        vbox.addWidget(self.b3)

        self.b1.clicked.connect(self.on_button_clicked)
        self.b2.clicked.connect(self.on_button_clicked)
        self.b3.clicked.connect(self.on_button_clicked)

        self.setLayout(vbox)

    def on_button_clicked(self):
        btn = self.sender()
        if btn == self.b1:
            print("btn1 clicked")
        elif btn == self.b2:
            print("btn2 clicked")
        else:
            print("btn3 clicked")
```

*Implementation of .sender() Method*



*GUI Displayed*

Python, like most coding languages, is constantly evolving with new modules being added and existing modules being modified. This handout provides a basic understanding; however, true mastery requires practice and your own exploration of what python has to offer.

# Displaying Tabular Data

As engineers, we often want to display data that follows a tabular format. With GUIs, we can display tabular data from various sources like CSV files, Excel files, and databases. PyQt5 offers several classes for us, but specifically we will explore the `QAbstractTableModel` and `QTableView` class.

## QAbstractTableModel

**QAbstractTableModel** is a class that provides a standard interface for presenting tabular data to a view, such as a **QTableView**. It is one of the model/view classes that come as part of the PyQt5 module

To use **QAbstractTableModel**, you will need to subclass it and provide implementations for several required functions. These functions include:

- `rowCount()`: returns the number of rows in the model
- `columnCount()`: returns the number of columns in the model
- `data()`: returns the data for a given model index
- `headerData()`: returns the data for the headers of the model

You will also need to implement the `setData()` function if you want to allow the user to edit the data in the model.

Here is an example of how to use the `QAbstractTableModel` to display data in a PyQt5 table view:

```
class TableModel(QtCore.QAbstractTableModel):

    def __init__(self, data):
        super(TableModel, self).__init__()
        self._data = data

    def data(self, index, role):
        if role == Qt.DisplayRole:
            value = self._data.iloc[index.row(), index.column()]
            return str(value)

    def rowCount(self, index):
        return self._data.shape[0]

    def columnCount(self, index):
        return self._data.shape[1]

    def headerData(self, section, orientation, role):
        if role == Qt.DisplayRole:
            if orientation == Qt.Horizontal:
                return str(self._data.columns[section])

            if orientation == Qt.Vertical:
                return str(self._data.index[section])
```

Python, like most coding languages, is constantly evolving with new modules being added and existing modules being modified. This handout provides a basic understanding; however, true mastery requires practice and your own exploration of what python has to offer.

## QTableView

**QTableView** is a widget that displays tabular data in a grid-like format. It is used to present data in a two-dimensional table view, with rows and columns.

Here is an example of how to use **QTableView** to display data in a PyQt5 application:

```
app = QApplication(sys.argv)

# Create some sample data
data = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Create the model
model = TableModel(data)

# Create the table view and set the model
table_view = QTableView()
table_view.setModel(model)

# Show the table view
table_view.show()

sys.exit(app.exec_())
```

This will create a table view with 3 rows and 3 columns, and display the sample data in it.

You can customize the appearance and behavior of the table view by setting its various properties, such as `horizontalHeader`, `verticalHeader`, and `selectionBehavior`. You can also handle events, such as cell clicks and double clicks, by connecting signals to slots.

## General Outline

1. Define a class to serve as a window that extends the `QDialog` class
2. Create an object from a class that extends `QAbstractTableModel`
  - a. For example construct a `SimpleTableModel` instance
  - b. Ensure that the parameters follow the correct format
3. Create a `QTableView` object
4. Embed the table model object to the `QTableView` object using the `setModel` method
5. Add the `QTableView` object to the desired layout

Below is a simple example assuming the `SimpleTableModel` class is defined and proper imports exist.

Python, like most coding languages, is constantly evolving with new modules being added and existing modules being modified. This handout provides a basic understanding; however, true mastery requires practice and your own exploration of what python has to offer.

# Documentation References

Here's a list of classes native to the PyQt5 library that are relatively popular and may be useful for customizing applications. Please reference the documentation links below for further details.

- [PyQt5.QtWidgets](#)
  - [QLabel](#) - Creates a label that displays text
  - [QLineEdit](#) - Allows users to enter text
  - [QComboBox](#) - Drop down list
  - [QCheckBox](#) - Creates a check box
  - [QPushButton](#) - Creates a push button
  - [QRadioButton](#) - Creates a radio button
  - [QMessageBox](#) - Creates a pop up window to display a message
  - [QListWidget](#) - Creates a list of text (Simpler, but less extensible than QListView)
  - [QListView](#) - Creates a list of text
  - [QDialog](#) - A top-level window mostly used for brief communications
  - [QTableView](#) - Creates a view of a table of rows and column data
  - [QAbstractItemView](#) - Used to set selection mode and more
  - [QAction](#) - Keyboard shortcuts and more
- [PyQt5.QtCore](#)
  - [QTime](#) - Self explanatory
  - [QDate](#) - Self explanatory
  - [QDateTime](#) - Self explanatory
  - [QTimeZone](#) - Self explanatory
  - [QAbstractTableModel](#) - Used to create SimpleTableModel
  - [QVariant](#) - Helper class for QAbstractTableModel
  - [Qt](#) - Helpful for keyboard shortcuts and more
- [PyQt5.QtGui](#)
  - [QKeySequence](#) - Keyboard shortcuts and more
  - [QStandardItemModel](#) - Useful for views
  - [QStandardItem](#) - Useful for QStandardItemModel

Python, like most coding languages, is constantly evolving with new modules being added and existing modules being modified. This handout provides a basic understanding; however, true mastery requires practice and your own exploration of what python has to offer.